



Tecnológico de Monterrey

Escuela de Ingeniería y Ciencias

Campus Monterrey

Nombre del trabajo:

Administración de Reto - Etapa 4
GreenWaterTech

Curso:

Implementación del internet en las cosas

Estudiantes:

Maria Sophia Valdez Armenta - A01253712

Carlos Humberto Martinez - A01281715

Fernando Perez Garcia - A01285236

Marco Antonio Lucio Sosa - A01285589

Rosendo De los Ríos - A01198515

Fecha de entrega:

19 de noviembre

Introducción

GreenWaterTech emerge como una respuesta innovadora a los desafíos críticos que enfrenta la agricultura, especialmente en términos de gestión eficiente del recurso hídrico. En un mundo donde la escasez de agua y la necesidad de prácticas sostenibles son imperativos, este proyecto se erige como un faro de cambio. La integración de tecnologías avanzadas, como sensores de humedad del suelo, Internet de las Cosas (IoT) y algoritmos inteligentes, busca no solo mejorar la eficiencia en el riego, sino también contribuir a los Objetivos de Desarrollo Sostenible. A través de la optimización de recursos y la reducción del desperdicio de agua, GreenWaterTech tiene el potencial de transformar la forma en que abordamos la agricultura y su impacto en el medio ambiente.

Desarrollo

Alcances del Proyecto:

GreenWaterTech tiene como objetivo revolucionar la gestión del riego en la agricultura y campos mediante la integración de una plataforma de Internet de las Cosas (IoT). El proyecto abarca desde la implementación de sensores de humedad del suelo hasta la creación de algoritmos avanzados de control y la aplicación de sistemas automatizados de riego. Además, se prioriza la experiencia del usuario mediante el desarrollo de una interfaz intuitiva, tanto en forma de aplicación móvil como en una interfaz web.

Área de Impacto:

La iniciativa impacta directamente en el sector de la agricultura, abarcando campos de cultivo y entornos residenciales. La implementación de prácticas sostenibles en el riego no solo beneficia la eficiencia en la producción de alimentos, sino que también tiene un impacto positivo en la conservación de recursos naturales.

Objetivos y Metas de Desarrollo Sostenible:

GreenWaterTech se alinea con los Objetivos de Desarrollo Sostenible (ODS) 11 y 12. En el caso del ODS 11, se busca contribuir a la creación de comunidades más sostenibles mediante la optimización de sistemas de riego. Asimismo, el ODS 12 es abordado al promover la producción y consumo responsables, especialmente en el contexto de la agricultura de regadío, donde la eficiencia en el uso del agua es esencial.

Descripción/Definición del Proyecto:

El proyecto se erige como una solución integral para combatir el desperdicio de recursos naturales, centrándose principalmente en el agua utilizada en la agricultura. La combinación de sensores de humedad y temperatura del suelo, conectados a una plataforma de IoT, permite la recopilación de datos en tiempo real. Estos datos, junto con la aplicación de algoritmos inteligentes, se utilizan para optimizar el proceso de riego, reduciendo significativamente el desperdicio y mejorando la eficiencia.

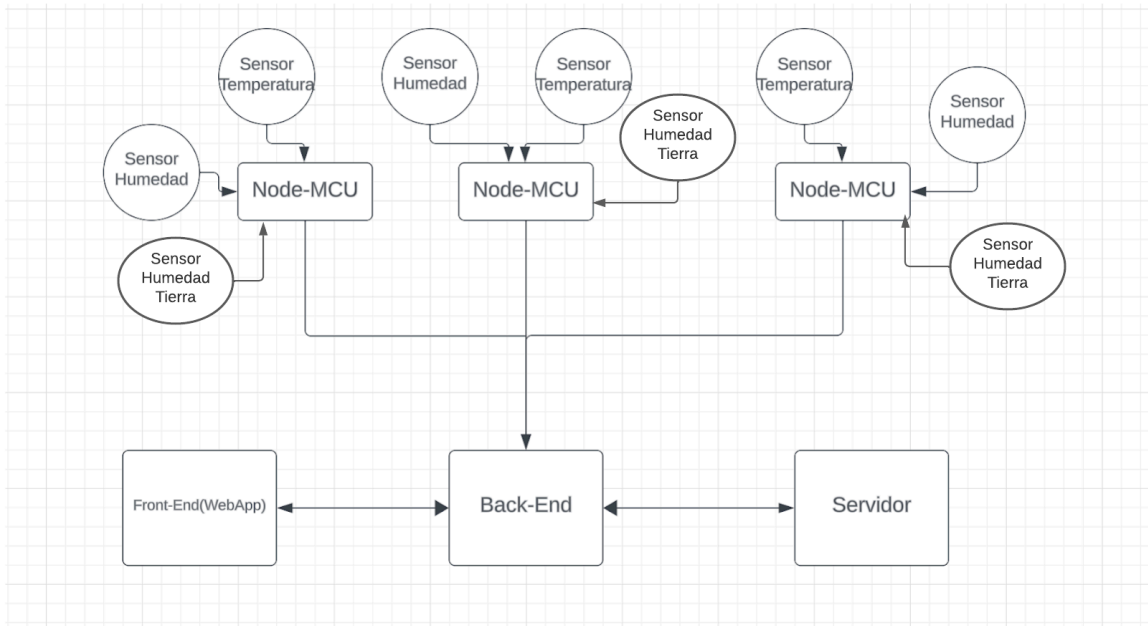
Problema a Resolver:

El desperdicio de agua en la agricultura, resultado de prácticas ineficientes de riego, es un problema crítico abordado por GreenWaterTech. Además de afectar la disponibilidad de un recurso vital, el agua, estas prácticas generan costos elevados para los agricultores, tienen impactos ambientales negativos y afectan la calidad del suelo y de los cultivos.

Beneficios del Proyecto:

Los beneficios derivados de la implementación de GreenWaterTech son numerosos. Desde un ahorro sustancial de agua al evitar el riego excesivo hasta mejoras significativas en la salud de las plantas y la calidad de los cultivos. Este enfoque no solo se traduce en ahorros monetarios para los agricultores, sino que también contribuye a la sostenibilidad ambiental al promover un uso responsable del agua en la agricultura.

Diagrama de la aplicación:



Base de Datos: Firebase

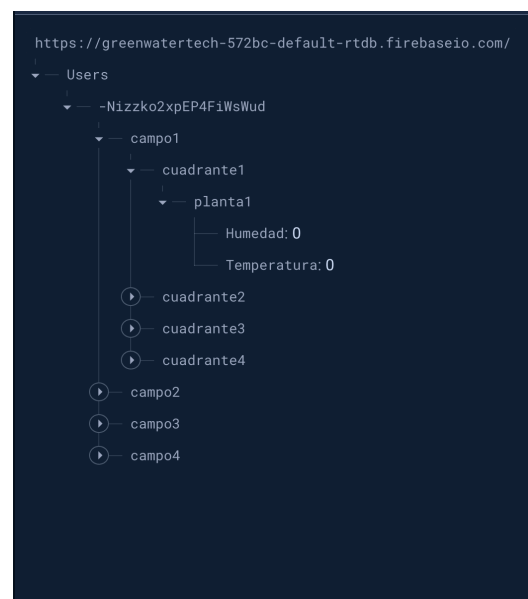
Para nuestro proyecto decidimos implementar una base de datos en la nube, de esta manera nos ahorramos problemas de manejos de IP y podrá estar siempre disponible. Nos decidimos por usar Firebase porque esta es una base de datos a base de diccionarios con alta flexibilidad que es compatible con JSON; como nosotros usamos Next en nuestro dashboard también nos facilita ya que al fin y al cabo es Java Script.

Indicadores a obtener de la base de datos:

- Temperatura y humedad ambiente en rangos de 30 minutos, 6 horas y 12 horas.
- Especie de la plantación en el cuadrante para tomar decisiones acorde.

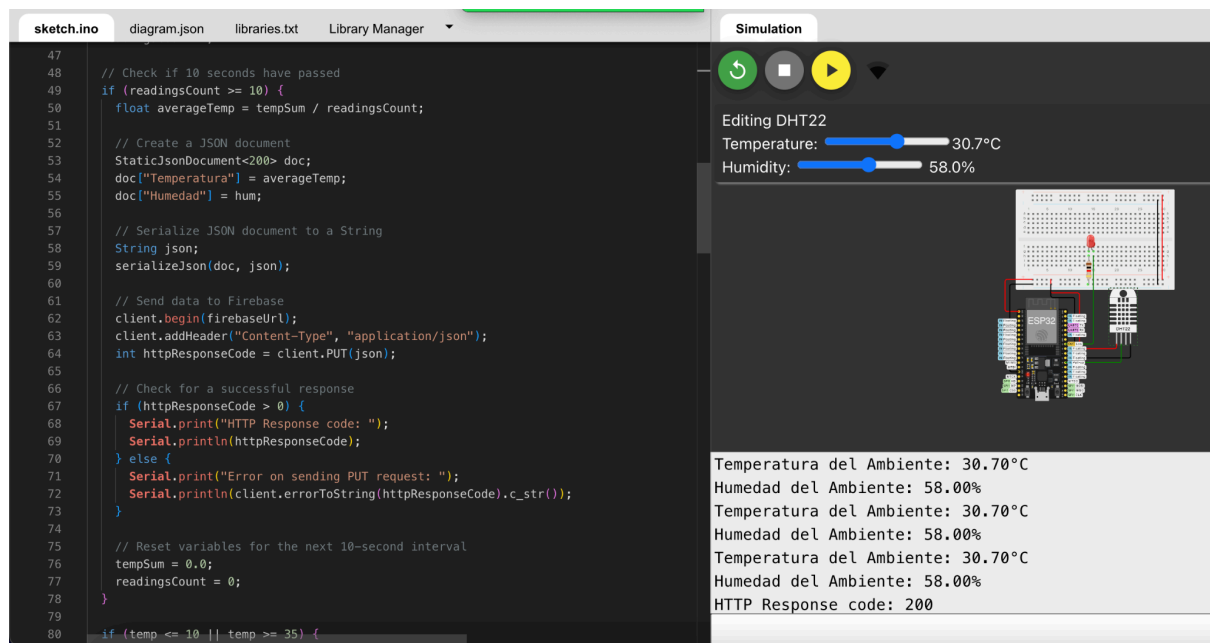
Estructura:

En la base de datos, cada vez que se crea un usuario se realiza el post al realtime database, donde se genera un ID único de usuario y se crean los campos con sus respectivos cuadrantes. De esta forma, cada cliente puede tener varios campos, de los cuales pueden tener una cantidad definida de 4 cuadrantes. Cada cuadrante guarda los datos que describimos anteriormente tanto



para la humedad como la temperatura. Estos son paulatinamente actualizados por el Backend con los datos de los nodos.

Conexión a base de datos:



Código de wokwi (simulación)

```
#include "DHTesp.h"
#include <ArduinoJson.h>
#include <WiFi.h>
#include <HTTPClient.h>
```

```
DHTesp dht;
```

```
float temp, hum;
float tempSum = 0.0;
int readingsCount = 0;
const int ledPin = 19; // Define el pin del LED
const char* ssid = "Wokwi-GUEST";
const char* password = "";
const char* firebaseUrl = "https://greenwatertech-572bc-default-rtdb.firebaseio.com/Users/-Niurw-W5-ayTImFS9yQ/camp01/cuadrante1/planta1/.json";
```

```
HTTPClient client;
```

```
void setup() {
```

```

    Serial.begin(9600);
    dht.setup(16, DHTesp::DHT22);    // Configura el pin y el tipo de
sensor
    pinMode(ledPin, OUTPUT);        // Configura el pin del LED como
salida

    // Connect to WiFi
    Serial.print("Connecting to WiFi");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("Connected to WiFi");
}

void loop() {
    hum = dht.getHumidity();
    temp = dht.getTemperature();

    Serial.print("Temperatura del Ambiente: ");
    Serial.print(temp);
    Serial.println("°C");
    Serial.print("Humedad del Ambiente: ");
    Serial.print(hum);
    Serial.println("%");

    // Calculate the average temperature over 10 seconds
    tempSum += temp;
    readingsCount++;

    // Check if 10 seconds have passed
    if (readingsCount >= 10) {
        float averageTemp = tempSum / readingsCount;

        // Create a JSON document
        StaticJsonDocument<200> doc;
        doc["Temperatura"] = averageTemp;
        doc["Humedad"] = hum;

        // Serialize JSON document to a String
        String json;
        serializeJson(doc, json);
    }
}

```

```

// Send data to Firebase
client.begin(firebaseUrl);
client.addHeader("Content-Type", "application/json");
int httpResponseCode = client.PUT(json);

// Check for a successful response
if (httpResponseCode > 0) {
    Serial.print("HTTP Response code: ");
    Serial.println(httpResponseCode);
} else {
    Serial.print("Error on sending PUT request: ");
    Serial.println(client.errorToString(httpResponseCode).c_str());
}

// Reset variables for the next 10-second interval
tempSum = 0.0;
readingsCount = 0;
}

if (temp <= 10 || temp >= 35) {
    digitalWrite(ledPin, HIGH); // Enciende el LED si la temperatura
    es menor o igual a 10°C o mayor o igual a 35°C
} else {
    digitalWrite(ledPin, LOW); // Apaga el LED en otros casos
}

delay(1000);
}

```

Este código está diseñado para ser ejecutado en una plataforma de simulación de hardware llamada Wokwi. La simulación está destinada a un dispositivo Nodemcu que utiliza un sensor DHT22 para medir la temperatura y la humedad ambiental. Luego, estos datos se envían a una base de datos en tiempo real de Firebase.

Librerías utilizadas:

- DHTesp.h: Librería para el sensor DHT22.
- ArduinoJson.h: Librería para manipular datos en formato JSON.
- WiFi.h: Librería para la conexión Wi-Fi.
- HTTPClient.h: Librería para realizar solicitudes HTTP.

Configuración inicial en el método setup():

- Se inicializa la comunicación serial con una velocidad de 9600 baudios.
- Se configura el sensor DHT22 en el pin 16.
- Se configura el pin 19 como salida para controlar un LED.

- Se establece la conexión Wi-Fi utilizando las credenciales proporcionadas (ssid y password).

Bucle principal en el método loop():

- Se obtienen las lecturas de temperatura y humedad del sensor DHT22.
- Se imprimen los valores en el monitor serial.
- Se acumulan las lecturas de temperatura durante 10 segundos para calcular un promedio.
- Si han pasado 10 segundos, se crea un objeto JSON con el promedio de temperatura y humedad.
- Se serializa el objeto JSON en una cadena.
- Se envía la cadena JSON a la base de datos de Firebase mediante una solicitud HTTP PUT.
- Se comprueba la respuesta de la solicitud y se imprime en el monitor serial.
- Se reinician las variables de acumulación para el próximo intervalo de 10 segundos.
- Se enciende un LED si la temperatura es menor o igual a 10°C o mayor o igual a 35°C; de lo contrario, se apaga el LED.
- Se añade un retardo de 1 segundo antes de comenzar el siguiente ciclo del bucle principal.

Código de Arduino (real)

```
#include <ESP8266WiFi.h>
#include <Firebase_ESP_Client.h>
#include <DHT.h>

// Provide the token generation process info.
#include <addons/TokenHelper.h>

// Provide the RTDB payload printing info and other helper functions.
#include <addons/RTDBHelper.h>

#define DHTPIN D3    // Replace with your actual DHT sensor pin
#define DHTTYPE DHT11 // DHT 11

DHT dht(DHTPIN, DHTTYPE);

// WiFi credentials
const char* ssid = "Tec-Contingencia"; // Replace with your WiFi SSID
const char* password = ""; // Replace with your WiFi password

// Firebase credentials
const char* firebaseHost = "greenwatertech-572bc-default-rtdb.firebaseio.com";
const char* firebaseAuth = "eNAbveZCFIoMyKLjIKp9UwXHsr4dJDt7pnV1dfe0"; // Replace with
your Firebase auth token

// Define Firebase Data object
```

```

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

const int ledPin = D1;

void setup() {
  Serial.begin(9600);
  dht.begin();

  // Connect to WiFi
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("Connected to WiFi");

  // Assign the project host and API key
  config.host = firebaseHost;
  config.signer.tokens.legacy_token = firebaseAuth;

  // Assign the callback function for the long running token generation task
  config.token_status_callback = tokenStatusCallback; // See addons/TokenHelper.h

  // Initialize Firebase
  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);

  // Set the LED pin as an output
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Read temperature and humidity
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  if (isnan(hum) || isnan(temp)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  // Read the soil moisture value, which should be connected to A0
  int soilMoisturePin = analogRead(A0);
  soilMoisturePin = map(analogRead(soilMoisturePin), 1023, 0, 0, 100);

```



```

// Log to the Serial Monitor
Serial.print("Temperature: ");
Serial.print(temp);
Serial.print("°C, Humidity: ");
Serial.print(hum);
Serial.print("%, Soil Moisture: ");
Serial.print(soilMoisturePin);
Serial.println("%");

// Control the LED based on soil moisture
if (soilMoisturePin < 30) {
    digitalWrite(ledPin, HIGH); // Turn on the LED if soil moisture is less than 30
} else {
    digitalWrite(ledPin, LOW); // Turn off the LED otherwise
}

// Send data to Firebase
if (Firebase.ready()) {
    // Set the path and the value
    if (Firebase.RTDB.setFloat(&fbdo,
        "-Nj8Y4gCCGdasz5SxLdp/campo1/cuadrante1/planta1/Temperatura", temp) &&
        Firebase.RTDB.setFloat(&fbdo,
        "-Nj8Y4gCCGdasz5SxLdp/campo1/cuadrante1/planta1/Humedad", hum) &&
        Firebase.RTDB.setInt(&fbdo,
        "-Nj8Y4gCCGdasz5SxLdp/campo1/cuadrante1/planta1/Humedad_Tierra", soilMoisturePin)) {
        // Success, do something, e.g., print the result
        Serial.println("Data sent to Firebase");
    } else {
        // Failed, print the error code
        Serial.print("Error sending data: ");
        Serial.println(fbdo.errorReason());
    }
}

// Wait a little before sending new data
delay(15000);
}

```

Este código está diseñado para ser ejecutado en una placa ESP8266 y utiliza diversas bibliotecas, como la librería ESP8266WiFi para la conexión a Wi-Fi, la librería Firebase_ESP_Client para la comunicación con la base de datos en tiempo real (RTDB) de Firebase, y la librería DHT para la lectura de datos de un sensor DHT11 que mide temperatura y humedad.

Configuración de pines y sensores:

- Se define el pin al que está conectado el sensor DHT.
- Se especifica el tipo de sensor DHT (DHT11 en este caso).
- Se crea una instancia del objeto DHT.

Credenciales de Wi-Fi y Firebase:

- Se especifican las credenciales de la red Wi-Fi a la que se conectará el dispositivo.
- Se proporciona la información de autenticación de Firebase, incluyendo el host y el token de autenticación.

Inicialización y conexión:

- Se inicia la comunicación serial para la depuración.
- Se inicia la conexión a la red Wi-Fi.
- Se espera hasta que se establezca la conexión.

Configuración de Firebase:

- Se configuran los parámetros necesarios para la conexión a Firebase.
- Se asigna una función de devolución de llamada para el estado del token.

Configuración del pin del LED:

- Se establece el pin del LED como salida.

Bucle principal (loop):

- Se lee la temperatura y humedad del sensor DHT.
- Se lee el valor de humedad del suelo de un sensor conectado al pin A0.
- Se imprime la información recopilada en el monitor serial.
- Se controla un LED en función del valor de humedad del suelo.
- Se envían los datos a Firebase en la ruta especificada.
- Se manejan errores de envío a Firebase.
- Se espera un tiempo antes de recopilar y enviar nuevos datos (15 segundos en este caso).

```
20:15:22.625 -> Data sent to Firebase
20:15:37.676 -> Temperature: 23.70°C, Humidity: 64.00%, Soil Moisture: 0%
20:15:37.942 -> Data sent to Firebase
20:15:52.949 -> Temperature: 23.80°C, Humidity: 63.00%, Soil Moisture: 0%
20:15:53.212 -> Data sent to Firebase
20:16:08.267 -> Temperature: 24.00°C, Humidity: 63.00%, Soil Moisture: 100%
20:16:08.531 -> Data sent to Firebase
20:16:23.574 -> Temperature: 23.90°C, Humidity: 63.00%, Soil Moisture: 100%
20:16:23.846 -> Data sent to Firebase
20:16:38.851 -> Temperature: 23.90°C, Humidity: 65.00%, Soil Moisture: 100%
20:16:39.113 -> Data sent to Firebase
20:16:54.153 -> Temperature: 23.90°C, Humidity: 64.00%, Soil Moisture: 100%
20:16:54.412 -> Data sent to Firebase
```

En esta foto se muestra la consola con los datos del código que está diseñado para monitorear la temperatura, humedad y humedad del suelo, y enviar estos datos a Firebase para su almacenamiento y análisis.

Datos desde el Node MCU:



Base de datos actual

Los datos que se recolectaron en la simulación se pueden ver en nuestra base de datos en la captura de pantalla anterior. La versión final de nuestro proyecto usará este mismo método para guardar los valores en la base de datos.

Indicadores a obtener de la base de datos:

- Temperatura y humedad ambiente en rangos de 30 minutos, 6 horas y 12 horas.
- Temperatura de la tierra en rangos de 30 minutos, 6 horas y 12 horas.
- Especie de la plantación en el cuadrante para tomar decisiones acorde.

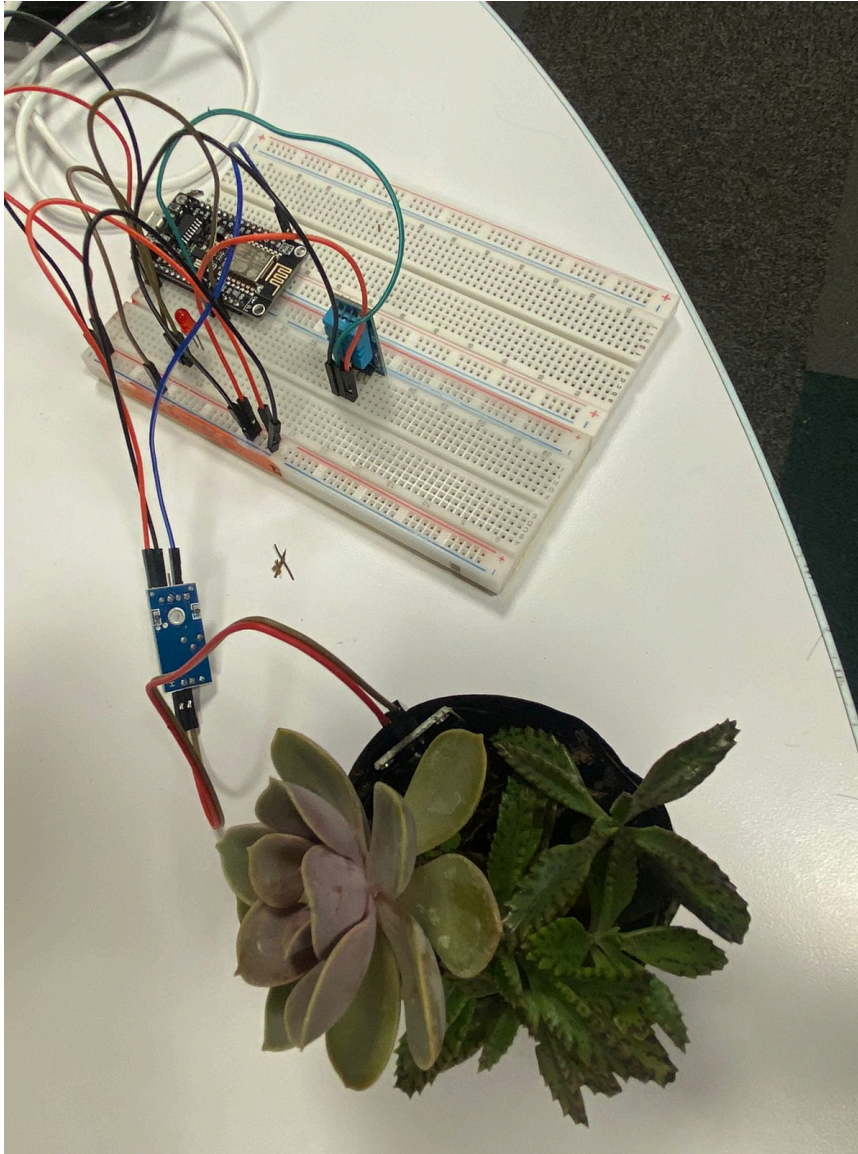
Modelo Físico:

El modelo físico implica la conexión de componentes a Arduino.

1. Sensor DHT11: Debe estar conectado al pin D3 en tu Arduino (como se especifica en 'DHT dht(D3, DHT11);').

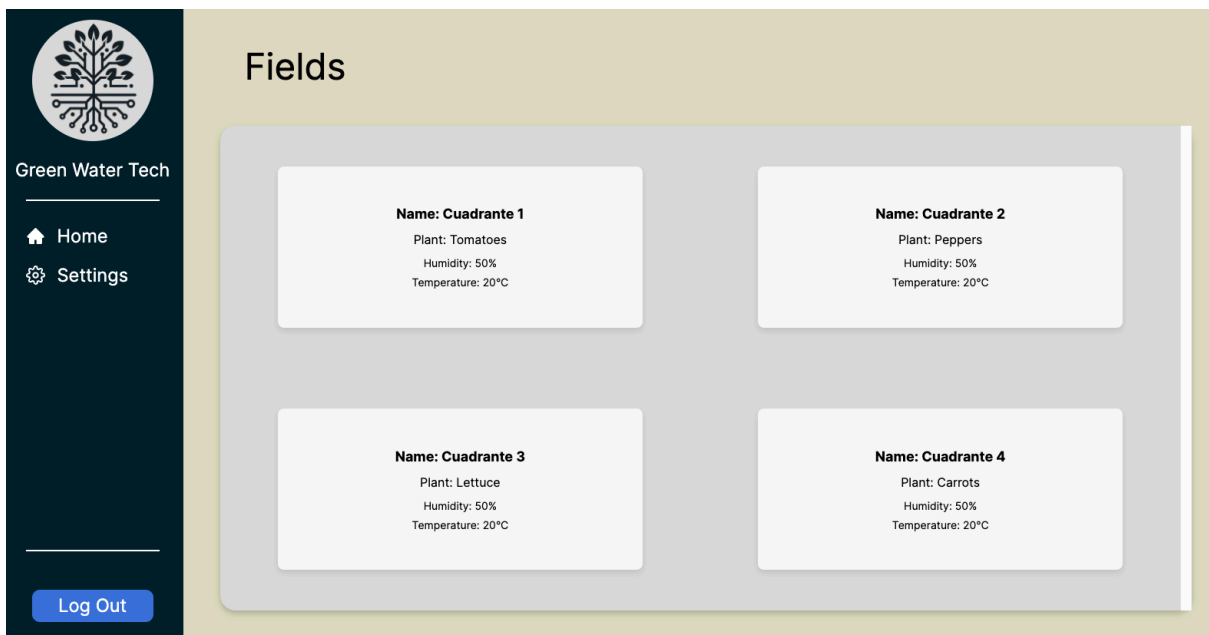
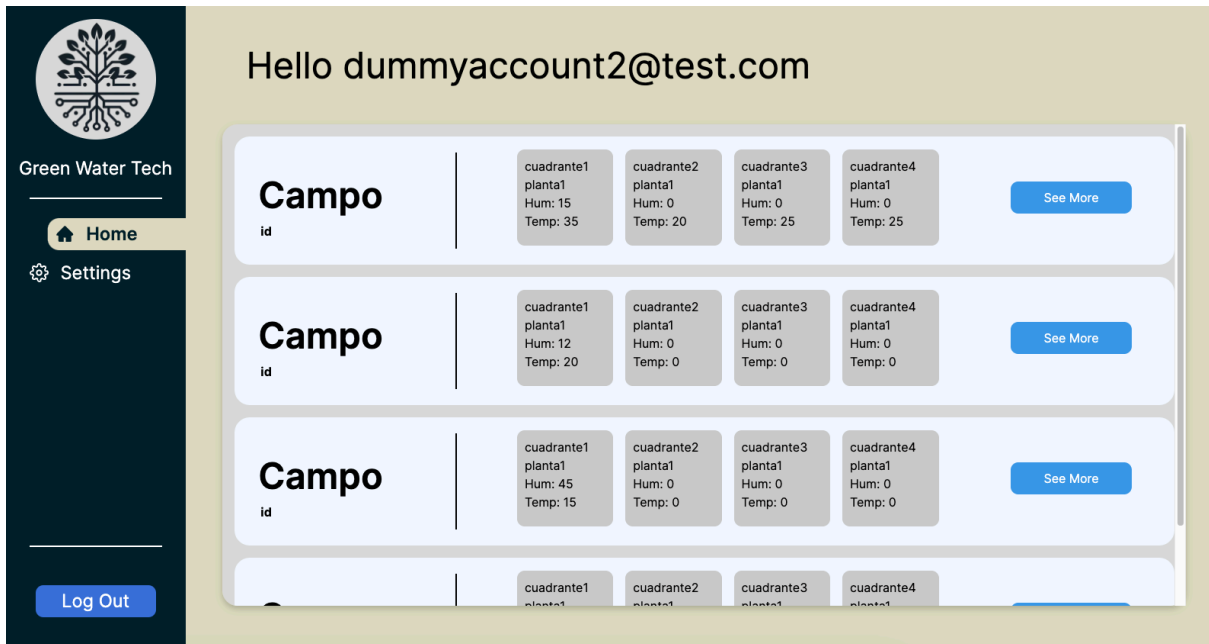
2. Sensor de humedad del suelo: Debe estar conectado al pin analógico A0 en tu Arduino (como se especifica en `int soilMoisturePin = A0;`). Asegurándonos de que el sensor de humedad del suelo esté calibrado correctamente y que sus conexiones se hayan realizado adecuadamente.

3. LED: Debe estar conectado al pin digital 13 en tu Arduino (como se especifica en `int ledPin = 13;`). El LED se enciende cuando la humedad del suelo es inferior a 30 y se apaga cuando la humedad es mayor o igual a 30.

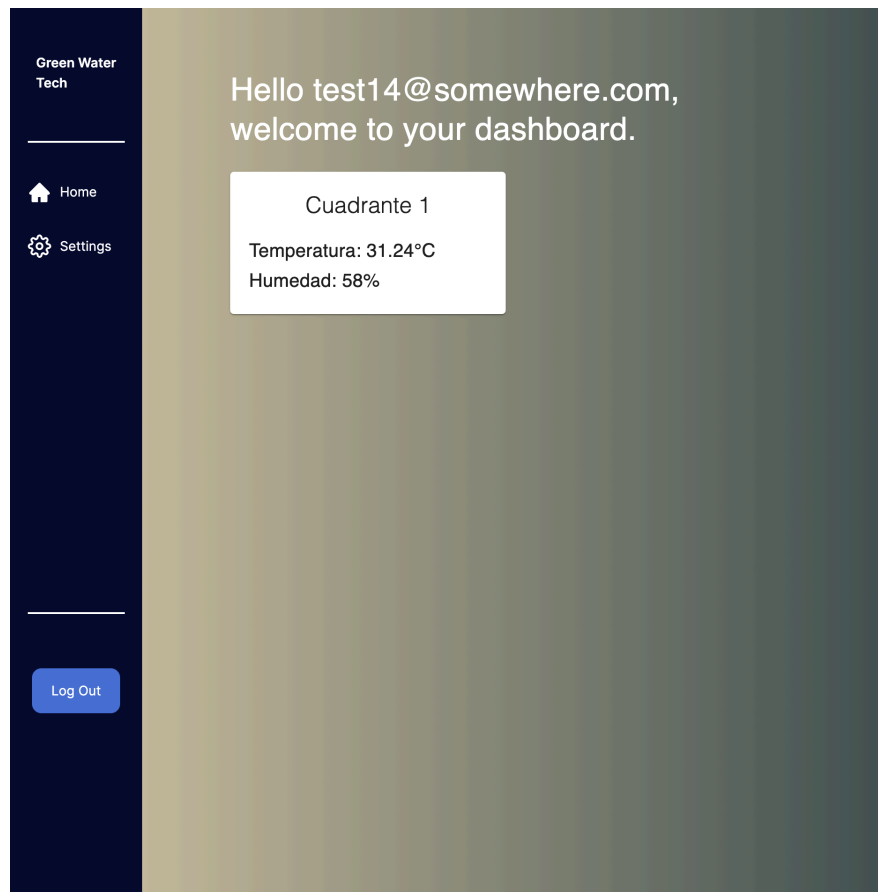


Web App:

Las siguientes capturas de pantalla muestran el estado actual 2 de nuestras páginas dentro de la aplicación web. Esta será la manera de que los usuarios puedan revisar toda la información de nuestra base de datos. Actualmente ya está conectada y se actualiza con los cambios de la Real Time database sin embargo todavía faltan algunos detalles de diseño.



Prototipo actual de dashboard funcional mostrando los datos de el servidor en tiempo real



Prototipo pasado de dashboard funcional mostrando los datos de el servidor en tiempo real

Próximos Pasos:

Los pasos siguientes incluyen la finalización del desarrollo de la interfaz de usuario, pruebas extensivas del sistema en condiciones simuladas y reales, y la implementación de la documentación (video y documentos). Se espera que GreenWaterTech, una vez plenamente operativo, sirva como modelo para prácticas sostenibles en la agricultura.

Conclusión

En conclusión, GreenWaterTech no es simplemente un proyecto técnico, sino una manifestación tangible de cómo la innovación puede catalizar un cambio significativo en la agricultura y la sostenibilidad ambiental. Al abordar el problema del desperdicio de agua en la agricultura, este proyecto no solo busca mejorar la eficiencia de los procesos, sino también establecer un estándar para la producción agrícola responsable. Con sus beneficios evidentes, desde el ahorro de agua hasta la reducción de costos y la mejora de la calidad de los cultivos, GreenWaterTech se posicionará como un actor clave en la construcción de un futuro agrícola más sostenible. Este proyecto no solo tiene el potencial de transformar campos y cultivos, sino también de inspirar iniciativas similares en todo el mundo, contribuyendo así a un futuro más sostenible y resiliente.

Video:

https://drive.google.com/file/d/1zeimVp8KL6EWI_dNGk9I72SwEk21C5eM/view?usp=sharing